BLOCCO 3 · LEZIONE 11

Strategic DDD avanzato: Subdomains, Context Mapping e confini di integrazione

Dispensa di studio • Durata orientativa 35–45 minuti • Materiale framework-independent

Obiettivi

- Distinguere Problem Space e Solution Space.
- Classificare Core, Supporting e Generic Subdomains.
- Individuare confini semantici, relazioni e ownership.
- Scegliere i context mapping patterns in base ai tradeoff.
- Produrre una specifica agent-ready con criteri di verifica indipendenti.

01 · La domanda architetturale

Un sistema puo avere controller, service e repository ben organizzati e tuttavia non sapere dove terminino le responsabilita di Operations e dove inizino quelle di Billing. Il punto di partenza sono le distinzioni del business, non quelle imposte dal framework.

- Come dividere un sistema complesso prima di far lavorare agenti AI?
- Le directory tecniche non esprimono da sole ownership e linguaggio.
- Strategic DDD identifica valore, boundary, dipendenze e responsabilità.

02 · Strategic vs Tactical DDD

Strategic e Tactical DDD non sono alternative. Il primo ragiona sulle aree del problema, sui modelli e sulle relazioni. Il secondo introduce strutture e regole dentro uno specifico bounded context. La Lezione 12 approfondira il livello tattico.

- Strategic: subdomains, bounded contexts, context map e team.
- Tactical: entity, value object, aggregate, domain service ed eventi.
- Prima definiamo chi possiede quale modello, poi come realizzarlo.

03 · Subdomain nel Problem Space

Nel gestionale di manutenzione, se il vantaggio consiste nel coordinare interventi e SLA, la pianificazione avanzata potrebbe essere Core; reportistica e anagrafiche Supporting; autenticazione e consegna email Generic. Sono classificazioni esemplificative: serve una decisione motivata dai business goal. Un componente Generic puo comunque essere critico.

- Core: dove nasce il vantaggio competitivo.
- Supporting: necessari, spesso specifici, ma non differenzianti.
- Generic: capacita comuni acquistabili o riutilizzabili.

04 · Subdomain ≠ Bounded Context

Il subdomain e ricavato dall osservazione del business. Il bounded context appartiene alla modellazione della soluzione: delimita un linguaggio ubiquitario e le sue regole. Un contesto puo attraversare piu subdomain, o un subdomain puo avere piu modelli. Non confondere contesto con microservizio, database o repository.

- Subdomain: porzione del problema di business.
- Bounded Context: confine dove un modello ha significato coerente.
- La relazione non e obbligatoriamente uno-a-uno.
- Il deployment non definisce automaticamente il contesto.

Regola: non derivare un servizio per ogni subdomain. Traccia prima una mappa dei modelli e verifica il linguaggio che cambia.

05 · Una parola, tre modelli

La parola intervento non ha necessariamente un significato unico. Un modello universale rischia campi facoltativi, lifecycle incompatibili e logiche contraddittorie. Ogni contesto conserva il proprio linguaggio e scambia solo concetti concordati.

- Reporting: MaintenanceRequest, una richiesta da valutare.
- Operations: WorkOrder, lavoro assegnato ed eseguito.
- Billing: BillableService, prestazione economica.
- Evita un unico oggetto universale Intervention.

06 · Context Map

Upstream e il contesto che influenza un dipendente downstream attraverso modello o servizi. La relazione non coincide necessariamente con il verso della chiamata HTTP. Una mappa utile registra anche owner, versione dei contratti e incognite. Preferisci mappe piccole che rispondano a domande specifiche.

- Rappresenta contesti, proprietari e relazioni.
- Esplicita Upstream e Downstream, contratti e traduzioni.
- Mostra i punti in cui cambiare un modello influenza un altro.
- E un artefatto vivo, non un diagramma una tantum.

07 · Partnership

Un team che pianifica le attivita e uno che assegna i tecnici possono condividere una release di business, ma partnership dichiarate senza capacita organizzativa creano colli di bottiglia. Usarla quando i due team riescono a coordinarsi davvero.

- Dipendenza reciproca e obiettivi condivisi.
- Roadmap e rilascio coordinati tra team.

- Richiede collaborazione reale e gestione delle modifiche.

08 · Customer / Supplier

Reporting pubblica un evento RequestAccepted che Operations deve consumare. Operations negozia le informazioni necessarie. Non e autorizzato ad alterare gli stati interni del producer. Ownership e governance del contract breaking change devono essere esplicite.

- Upstream fornisce capacita e contratti.
- Downstream puo influenzare il servizio ricevuto.
- Il contratto e negoziato; il modello interno resta dell owner.

09 · Conformist

Un sottosistema di reportistica non differenziante puo recepire esattamente gli stati di un vendor. Il conformismo non e una debolezza automaticamente: e un compromesso consapevole. Se l upstream cambia, il downstream potrebbe dover seguire.

- Downstream adotta il modello di Upstream.
- Riduce traduzioni e tempo di integrazione.
- Aumenta accoppiamento e dipendenza semantica.
- Conveniente solo se il modello esterno e accettabile.

10 · Anti-Corruption Layer

Se il sistema esterno invia DONE, non possiamo assumere che il lavoro sia amministrativamente verificato e fatturabile. Un ACL puo tradurlo in externalCompletionReported. La decisione di fatturabilita spetta a Billing e alle sue policy confermate.

- Traduce il significato, non solo il formato.
- Protegge il linguaggio e gli invarianti interni.
- Valido per legacy, vendor e integrazioni non controllate.
- Non inventa equivalenze tra stati business.

```
ExternalJob(status="DONE")
   -> ACL -> externalCompletionReported = true
   != billable = true  // policy UNKNOWN
```

11 · Altri pattern

I pattern descrivono prospettive diverse: relazioni organizzative, esposizione di servizio e propagazione del modello. Non sono tutti mutuamente esclusivi. Big Ball of Mud descrive invece confini scarsamente affidabili che vanno contenuti.

- Shared Kernel: porzione di modello mantenuta congiuntamente.
- Open Host Service: interfaccia stabile verso piu consumer.
- Published Language: linguaggio concordato per lo scambio.

- Separate Ways: evitare integrazione non conveniente.

12 · Ownership dei confini

Gli agenti devono ricevere le responsabilita gia chiarite. Una tabella owner/consumer aiuta: Reporting possiede RequestStatus, Operations WorkOrderStatus, Billing le regole di BillableService. L owner del contratto pubblicato non autorizza a interpretazioni arbitrarie nel consumer.

- Model ownership: chi definisce linguaggio, stati e invarianti.
- Contract ownership: chi pubblica e governa interfacce.
- Data authority: chi e fonte autorevole di una informazione.
- Change governance: chi approva breaking change.

13 · Task agent-ready

Esempio: realizza il consumer dell evento WorkCompleted v1 in Billing senza importare persistenza Operations, validando schema e idempotenza solo dopo avere deciso la relativa policy. Il coding agent puo proporre opzioni, ma non decidere il significato dei dati non chiariti.

- Context + decisioni approvate + input + vincoli.
- Open questions preservate come UNKNOWN.
- Output: adapter, mapping, contract test, verifica.
- Non convertire WorkCompleted in Billable senza policy.

```
DECIDED: Operations possiede WorkOrderStatus
DECIDED: Billing possiede billability policy
UNKNOWN: completamento tecnico sufficiente?
FORBIDDEN: import diretti dei model Operations
```

14 · Failure mode

Il rischio non e solo tecnico: test e build possono risultare verdi pur avendo incorporato regole semanticamente errate. Riesaminare periodicamente mappa, contratti, codice e ownership.

- Un context per ogni tabella; ogni funzione e Core.
- Microservizi prima del domain modeling.
- Conformist involontario nel Core.
- ACL che inventa business policy.
- Mappe obsolete e boundary senza owner.

15 · Laboratorio progressivo

Scenario: segnalazione accettata, lavoro assegnato, tecnico segnala completamento; resta UNKNOWN se il completamento autorizza la fattura. Classifica subdomain; assegna owner; rappresenta flussi; proponi un pattern di integrazione; scrivi test dei divieti architetturali.

- Produci SUBDOMAINS.md e GLOSSARY.md.

- Crea CONTEXT-MAP.md e ADR-001.
- Bozza del contratto WorkCompleted v1, stato PROPOSED.
- Tre verifiche indipendenti dei boundary.

16 · Takeaway

Domain -> Requirements -> Spec -> Test -> Agent -> Verify. Le decisioni strategiche antecedono il coding e rendono possibile un orchestrazione multiagente controllabile. Fonti: Evans, Vernon, Martin Fowler, DDD-Crew, Context Mapper e Microsoft Architecture Center.

- Il dominio governa la forma dei confini.
- Il confine semantico non coincide con la topologia del deploy.
- Gli agenti implementano relazioni decise, non inventano policy.
- Prossima: Tactical DDD e Consistency Boundaries.

Laboratorio completo

Scenario

Una piattaforma riceve segnalazioni di manutenzione, assegna lavori e prepara dati di fatturazione. Un tecnico puo dichiarare lavoro concluso. NON sappiamo se questa dichiarazione sia sufficiente per fatturare: UNKNOWN.

A. Subdomain e confini

Elenca Core, Supporting e Generic motivando la classificazione alla luce del vantaggio competitivo ipotizzato. Definisci tre bounded context (Reporting, Operations, Billing) e, per ciascuno, glossario locale, responsabilita, fonte autorevole e policy esplicitamente sconosciute.

B. Context Map

Disegna due integrazioni: Reporting -> Operations, Operations -> Billing. Documenta per ciascuna producer, consumer, dipendenza upstream/downstream, pattern candidato, contratto, traduzioni e owner. Distingui le ipotesi dalle decisioni approvate.

C. Artefatti

Crea docs/domain/SUBDOMAINS.md, docs/domain/GLOSSARY.md, docs/domain/CONTEXT-MAP.md, docs/decisions/ADR-001-integration-boundaries.md e docs/contracts/work-completed-v1.schema.json in stato PROPOSED.

D. Verifiche

Test 1: nessun import di classi persistence di Operations nel codice Billing. Test 2: ogni evento entrante conforme al contratto versione dichiarata. Test 3: nessuna trasformazione automatica di WorkCompleted in autorizzazione alla fatturazione senza policy.

E. Accettazione

Il lavoro e completo se ogni decisione e tracciabile, gli UNKNOWN restano aperti, la mappa distingue responsabilita e relazioni, e almeno tre controlli possono essere svolti indipendentemente dall agente implementatore.

Autovalutazione

- Un subdomain coincide sempre con un bounded context? No: descrivono piani concettuali differenti.
- Conformist e sempre sbagliato? No: puo minimizzare il costo di integrazione quando e accettabile il modello upstream.
- Un ACL e solo mapping JSON? No: e una protezione semantica.
- Un evento di completamento garantisce fatturabilita? Non senza business policy esplicita.
- Context Map e un artefatto statico? No: va mantenuto allineato ai contratti e alle responsabilita.

Riferimenti e fonti

• Eric Evans, Domain-Driven Design (2003)

• Vaughn Vernon, Implementing Domain-Driven Design (2013); Domain-Driven Design Distilled (2016)

• https://martinfowler.com/bliki/BoundedContext.html

• https://virtualddd.com/ddd-crew/context-mapping/

• https://contextmapper.org/docs/language-reference/

• https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis